TRADEGAUNTLET: TESTING THE LIMITS OF TRADING COMPLIANCE CONTROLS

ADAPTIVE EVASION, ORDER SPLITTING, AND CANCELLATION TIMING

PREPRINT

Sandip Nandi
Independent Researcher

Aishwarya Bose
Independent Researcher

Apurba Bose
Independent Researcher

Soovadeep Bakshi
Independent Researcher

ABSTRACT

Pre-trade gates reject orders before they reach the market; surveillance rules flag suspicious order patterns afterwards. Both are usually judged by counting alerts. We argue that three things must be measured separately, on matched market paths: whether a rule alerts, whether a gate removes the gain from moving the price, and how much legitimate trading the gate blocks. TradeGauntlet, a replayable limit-order-book simulator, makes this possible. First, a detector that shares a threshold with a gate is silent on gated traffic by construction, so zero alerts there measure the gate. Second, with a price response fitted to Nasdaq order-book data for five stocks, a seeded search finds split-order schedules that evade the implemented conduct rules under every control we test, scored on held-out paths and also when only depth near the best price moves it. Replayed on real Nasdaq price paths, which are more volatile and heavy-tailed than the simulator's, the schedules keep their gains, but the ungated ones trigger alerts in about half the windows. Third, an aggregate cap at twice reference depth removes between none and about half of the gain, and an order-to-trade ratio and a learned detector together catch every searched schedule but flag 23% of a legitimate large maker's sessions. Fourth, legitimate makers also gain from the price response, and the cancellation throttle, not the cap, carries most of their cost. A paired check in ABIDES finds no detectable gain from displayed depth. Finally, with two LLMs trading legitimate tasks, every alert was a self-match by a market maker, which the gates remove by construction; the measurable effects are the gates' cost in fills and a prompt reminder that reduces Sonnet's self-matching. All trading is simulated.

Keywords trading compliance · market surveillance · spoofing · limit order book simulation · adversarial evaluation

1 Introduction

Consider a pre-trade gate that rejects a 60-lot order. The same gate may accept six 10-lot orders at the same price, because it checks each order on its own and misses their combined display. A cap on total resting quantity closes that gap. Whether the cap is worth having, though, depends on how the market responds to displayed orders and on how much legitimate trading the cap prevents.

This paper asks how to evaluate such controls against a trader who adapts to them. Two kinds of control are involved. A *gate* checks each order before it reaches the market and may reject it. A *detector*, or surveillance rule, inspects the order stream afterwards and raises an alert when it sees a pattern such as spoofing. The two are usually assessed separately, by counting alerts. An adaptive trader, however, can search for the gap between them, and a gate can silence a detector without removing the gain the detector was meant to catch. The question has become more pressing now that brokers let automated agents place and cancel orders on a customer's behalf [23, 24], but it applies to any adaptive trader. Most of our experiments use scripted and searched strategies; Section 10 adds two language-model agents given legitimate tasks.

We built TradeGauntlet for this study. It is a small limit-order-book simulator with a workflow for fixing scenarios in advance, comparing controls on identical market paths, and replaying every resulting order. It keeps three questions apart: does a surveillance rule flag a sequence, does a gate remove the sequence's gain from the simulated price response, and how much legitimate trading does the gate prevent? An alert records a pattern; a rejection records an intervention. Neither tells us on its own whether the intervention removed the gain, or whether the trading was lawful.

Our contributions are:

1. An evaluation method for stacks of trading controls. It scores alerts, removed gains, and legitimate execution separately on matched market paths, and replays every episode exactly. Two simple observations show when such an evaluation is uninformative by construction: a detector that shares a threshold with a gate cannot alert on gated traffic (Observation 1), and a step price response switches on at a closed-form cap multiple (Observation 2).
2. A price response fitted to Nasdaq order-book data for five stocks (Section 5). It turns the question of whether a cap protects into a measured quantity rather than an assumed one.
3. An adaptive adversary in place of hand-written counterexamples: a seeded search over split-order schedules, selected on training paths and scored on held-out paths with bootstrap confidence intervals (Section 6). We test it against a larger search budget, search restarts, the response window, the unit mapping, and real price paths, and compare it with two baseline detectors (Section 7).
4. Measurements of what the controls cost legitimate traders (Section 8), a check in the independent ABIDES simulator (Section 9), and a study of Claude Haiku 5.5 and Sonnet 5.5 trading under the same controls (Section 10).

The appendix adds mechanism checks under a stipulated step response, including a legitimate market maker that triggers a detector purely through its quoting interval. The simulator makes these interactions inspectable; it does not estimate how often they occur in real markets.

2 Related work

Wang and Wellman [1] study spoofing with trading agents, and Wang et al. [7] extend the analysis to strategic responses and market-design changes. Cartea, Jaimungal, and Wang [18] model the trade-off between spoofing gains and expected fines. Lee, Eom, and Park [34] measure the strategies and profits of spoofing traders in Korean market data, and Williams and Skrzypacz [35] show when spoofing arises in equilibrium. Wang, Vorobeychik, and Wellman [36] propose cloaking part of the order book, an intervention that, like our caps, limits how much displayed depth can move prices. These studies explain economic mechanisms that our fixed scripts and imposed price response take as given.

Tao et al. [19] derive spoofing conditions from a multilevel imbalance model, calibrate it to TMX Level-2 data, and propose monitoring based on distributional distance. Chen, Hsieh, and Yang [20] use Taiwan derivatives order and trade data to study potential spoofing and a price-banding intervention. Both bring market evidence to questions of detection or trading restrictions. We fit our response model to Nasdaq data for five stocks (Section 5); the rest of the comparisons measure effects within the simulator.

Two working papers use evidence that our design lacks. Do and Putniņš [13] evaluate indicators using prosecuted cases and out-of-sample tests. Fabre and Challet [10] use Level-3 data to estimate price responses and possible spoofing gains. We also fit a response to data, but we have no external labels of manipulation.

Learning traders can adapt to detection and enforcement. Byrd [2] studies an RL trader that discovers spoofing and a learned recognizer that discourages it. Wang and Wellman [3] model adaptation between manipulation and detection, while Cartea, Chang, and García-Arenas [12] examine manipulation by learning algorithms. Gu et al. [17] vary liquidity and spoofing strategies, and ABIDES [4] supports richer agent interaction and communication latency. Tuccella, Nadler, and Şerban [37] train a recurrent detector for spoofing on order-book data. Our adversary is simpler than these learners: it searches a fixed family of split-order schedules rather than learning a policy, which keeps each strategy it finds inspectable and replayable.

Realism metrics for limit-order-book simulators [38] and generative market simulators [39] measure whether a simulator reproduces market stylized facts. Our minimal book does not reproduce them on its own price paths, so we also replay real price paths (Section 6).

Pandey and Rajwal [25] evaluate LLM judgments about potential market abuse using variants of enforcement-case narratives. RISCTrade [26] evaluates adherence to risk mandates during trading across multiple markets. We focus on what happens at the order level when a gate or surveillance rule changes, including the cost to legitimate execution.

InvestorBench [27] compares LLM agents across financial tasks and assets. FinAgent [28] develops a multimodal trading agent with tools and memory. QFinZero [29] provides standardized financial tools, stateful brokerage simulation, order-lifecycle management, and replay. Here, we compare interacting controls on matched market paths, with scripted, searched, and LLM traders.

ReguSim [8] separates a trader's reasoning, attempted orders, enforcement, and monitoring evidence; its reported environment uses exogenous prices and fully executes accepted orders. Borjigin and He [9] propose constrained execution with a runtime shield, leaving the full cryptographic proof pipeline for future work. Kurshan, Balch, and Byrd [5] discuss agent-based governance, and Byrd [11] studies trading with LLM-generated social posts. Our LLM traders choose orders from the book alone, without a social-information channel.

3 TradeGauntlet: simulator, gates, and detectors

Some terms recur throughout. The *subject* is the account under test; everything else in the book is background liquidity or noise. The *touch* is the best bid or ask. *Reference depth* is the median external quantity at the two touches, excluding the subject's orders. Prices are integer *ticks*, and wealth is measured in *tick-lots*, one tick of price on one lot. A *session* or *episode* is one simulated trading period on one market *path*, the sequence of exogenous price and noise draws fixed by a seed.

3.1 Market mechanics

TradeGauntlet uses its own matching engine, written for this project rather than built on ABIDES, with integer prices, price/time priority, and partial fills. Scenario runners fix the configuration and random seeds, run paired control conditions, record attempted and accepted actions, and replay saved attempts to check event hashes and outcomes. The runners can vary cap multiples, response thresholds, throttle windows, session lengths, and quoting cadences. Orders execute only inside the simulator; there is no broker or exchange connection.

Component	Specification
Book and matching	One evaluated account and one instrument; integer ticks, price/time priority, partial fills
Timing	One second per attempted action; background replenishment every second; default decisions every 60 seconds, with exact re-quote times in the cadence boundary checks
Background liquidity	Three price levels per side, 10 lots per level, centered around the fundamental plus any response
Exogenous path	Fundamental starts at 1,000; increments -1, 0, +1 with probabilities 1/4, 1/2, 1/4 every 30 seconds; noise quantities 0, 1, 2 with probabilities 1/2, 1/4, 1/4 every 10 seconds
Risk and fees	100-lot worst-case position commitment; fee 0.05 tick-lots per filled lot
Sessions	720 seconds in the core study; 720 and 3,600 seconds in the throttle tests; no warm-up

Table 1: Simulator configuration.

The fundamental and noise streams are generated before trading starts. For a given seed and session length, every gate and response condition therefore receives the same exogenous draws. Unfilled market quantity expires. Background quotes are replaced between successive subject actions, so splitting an order also changes how long the display remains in the book.

The price response is driven by order-book imbalance. Immediately before each replenishment, let B and A be the total remaining buy and sell quantity, including the subject's. The imbalance is $I = (B - A)/\max(1, B + A)$. Under the *step* response, background quotes shift up by two ticks when I exceeds a threshold t, down by two ticks when I is below $-t$, and otherwise stay where they are. The default threshold is 0.4; Appendix A varies it. In the *unresponsive* condition the quotes never shift. The step response is stipulated, not estimated from market data.

Because every result under the step response hinges on a threshold we chose, we add a second response with no threshold. Under the *linear* response, background quotes move κ ticks in the direction of I with probability $|I|$ and otherwise stay centred on the fundamental, so the expected displacement is κI ticks. This follows two empirical regularities: the probability that the next price move is upward rises with order-book imbalance, roughly linearly for large-tick stocks [31], and short-horizon price changes are approximately linear in order-flow imbalance [30]. We use $\kappa = 1$ as a stress setting. The uniform draws that decide each move come from a separate seeded stream generated before trading, so all gate conditions on a seed share them. Section 5 fits the move probability to market data.

3.2 Pre-trade gates

Every condition enforces a bound on the account's worst-case position. The *per-order gate* (called legacy in the code) also rejects a limit order that is larger than twice reference depth, that would occupy a third resting price level on one side, or that would trade against one of the account's own resting orders (a self-match). The *aggregate gate* adds a cap per side: the subject's remaining limit quantity plus the full proposed quantity must not exceed m times reference depth. The default is $m = 2$. The sensitivity checks vary this aggregate multiple but keep the per-order limit at twice reference depth.

Both gates can also throttle the account: they reject new limit and market orders when $C/N > 0.9$ and $N \geq 20$, where C counts the subject's cancellations and N its accepted submissions in a trailing window of 1,800 seconds by default. Rejected attempts do not count towards N, and cancellations are always allowed. To switch the throttle off, we make the minimum count unreachable and leave the other checks unchanged. The aggregate check runs only at submission, so it does not revisit resting orders when external depth changes. It also checks a marketable limit order at its full quantity, which makes its rejection cost deliberately conservative.

3.3 Pattern definitions

For a canceled order i, let q_i be submitted quantity, d_i its submission-time external reference depth, and a_i its age at cancellation. Let h_i be the fraction of q_i filled before cancellation and z_i its distance from the same-side touch at submission. For a subject fill f on the opposite side, define $Q(i, f)$ to hold when i was nonmarketable at submission, submission precedes f, f precedes cancellation, $a_i \leq W$, and $h_i < 0.1$. Event sequence numbers determine ordering within a timestamp. Table 2 lists the implemented rules.

Signal	Condition for an alert
Individual pattern	Some i and f satisfy $Q(i, f)$, $q_i \geq 3d_i$, and $z_i \leq 3$ ticks.
Layer pattern	For one f and one displayed side, qualifying $Q(i, f)$ orders occupy at least three distinct prices. No individual-size or distance test is added.
Aggregate pattern	For one f and displayed side, at least two $Q(i, f)$ orders have $z_i \leq 3$ and positive remaining quantity $r_i(f)$. Their sum is at least $3 \max_i d_i$, three times the largest reference depth in that group.
Self-match	The subject is both buyer and seller in a fill.

Table 2: Conduct-alert patterns. Their union is the conduct-alert signal.

Remaining quantity $r_i(f)$ is measured immediately before the shared fill, after any earlier partial executions. The *conduct alert* is the union of these four signals; a separate cancellation-ratio diagnostic is not part of it. The scoring window is $W = 180$ seconds by default, shared by the individual, layer, and aggregate patterns. Table 3 keeps the 120-second window of the first split-order comparison, and Appendix A.3 varies W. These patterns are not a complete legal test and do not estimate intent.

3.4 Comparisons

Terminal wealth V is cash plus inventory valued at the terminal fundamental, after fees. For gate g, parameters p, and seed s, the response-dependent difference is

$$D(g, p, s) = V_{\text{resp}}(g, p, s) - V_{\text{unresp}}(g, p, s).$$

Both conditions use the same gate and exogenous path. Positive D means the strategy benefits from the price response. It is not a legal or welfare measure, and it is not the strategy's profit: a schedule can have $D > 0$ and still lose money overall. Executed lots, rejected actions by reason, and response activations are reported separately.

Our first test strategy is the *split probe*. Each cycle it submits six 10-lot orders at one price, executes two lots on the opposite side, and cancels whatever display survives; successive cycles alternate sides. The legitimate controls are a small and a large two-sided market maker, a patient maker, a passive scheduled executor, and a 10-lot slice executor. We assigned these scripts their objectives, and the assignments have not been independently reviewed. The scheduled executor is named `vwap_executor` in the code but uses neither a volume forecast nor a VWAP benchmark.

4 Gate–detector coupling

Table 3 runs the split probe on 40 paths. Each split order passes the individual-size check, and all of them sit at one price level, so the individual and layer detectors flag none of the 40 paths. The aggregate detector flags the sequence

Gate	Original alerts	With aggregate signal	Executed lots	Wealth
off	0/40	40/40	24.00 [24.00, 24.00]	22.05 [14.80, 32.80]
legacy	0/40	40/40	24.00 [24.00, 24.00]	22.05 [14.80, 32.80]
aggregate	0/40	0/40	24.00 [24.00, 24.00]	-25.95 [-33.20, -15.20]

Table 3: Six-by-ten split probe, `pilot.review_study`, seeds 3000-3039, responsive market. Both gates include the default throttle. The 120-second scoring window is retained from this comparison; rescoring at the current 180-second default leaves these alert counts unchanged. Alerts count episodes; volume and wealth are mean [minimum, maximum]. Wealth is in tick-lots.

whenever it is admitted. Under the aggregate gate, however, the absence of an aggregate alert says nothing about the detector. It follows from the inequality the gate and detector share.

Observation 1 (Gate–detector coupling)**.** *Suppose every order in a candidate group was admitted by the aggregate gate with multiple m, and the aggregate pattern alerts when the group's remaining quantity at the shared fill is at least $k \max_i d_i$. If $m < k$, the aggregate pattern never alerts on that group.*

Proof. Let j be the last group order admitted. At its admission the gate requires that the remaining same-side quantity, including q_j, be at most $md_j \le m \max_i d_i$. Orders admitted before j can only lose quantity afterwards, through fills or cancellation, and no order grows after admission, so the remaining quantity at the shared fill is also at most $m \max_i d_i < k \max_i d_i$. □

With $k = 3$ and the default $m = 2$, the zero alert count under the aggregate gate is guaranteed and says nothing about how accurate the detector is. More generally, when a gate and a detector threshold the same quantity, evaluating the detector on gated traffic measures the gate.

5 Fitting the price response

We use the five free LOBSTER samples: AAPL, AMZN, GOOG, INTC, and MSFT on 21 June 2012, reconstructed from Nasdaq messages at 10 levels (148,000 to 669,000 book updates per stock). AAPL, AMZN, and GOOG traded between about $220 and $580 with spreads of several ticks. INTC and MSFT traded near $27 and $30 and usually had a one-tick spread, so they are large-tick stocks in the sense of [31].

On a one-second grid, excluding the first and last ten minutes of the day, we regress the mid-price change over the next h seconds, in units of the prevailing half-spread, on the imbalance of the top three levels. Three levels match the simulator's three background levels. Intervals use a block bootstrap with five-minute blocks.

The fitted slope must be mapped into simulator units, and there are two reasonable ways to do it. The simulator's half-spread is one tick. If its spread stands for the market's spread, the one-second slope is directly comparable with κs, and we set the fitted scale s to a stock's one-second point estimate; this is the *half-spread mapping*, used unless stated otherwise. If instead a simulator tick stands for a market tick, the comparable scale is the slope times the median half-spread in ticks (the *tick mapping*). Section 6 reports results under both. AMZN and GOOG give slopes close to AAPL's, so the adversary uses AAPL, INTC, and MSFT as its fitted settings.

	Spread		Top three levels			Best level
Stock	(ticks)	Obs.	1 s	5 s	10 s	1 s
AAPL	15	18,028	0.018 [−0.003, 0.039]	−0.083 [−0.155, −0.009]	−0.150 [−0.279, −0.016]	0.052 [0.031, 0.075]
AMZN	12	15,340	0.032 [0.015, 0.049]	0.069 [0.007, 0.129]	0.063 [−0.044, 0.166]	0.036 [0.022, 0.050]
GOOG	26	11,152	0.016 [−0.005, 0.037]	0.028 [−0.039, 0.097]	0.052 [−0.062, 0.166]	0.025 [0.007, 0.044]
INTC	1	18,889	0.348 [0.287, 0.417]	1.169 [0.932, 1.406]	1.721 [1.321, 2.130]	0.201 [0.172, 0.229]
MSFT	1	19,220	0.479 [0.406, 0.574]	1.515 [1.270, 1.803]	2.041 [1.683, 2.476]	0.252 [0.222, 0.284]

Table 4: Slope of the future mid-price change, in half-spreads, on order-book imbalance, 21 June 2012. Spread is the median quoted spread; observations are one-second grid points. Brackets are 95% block-bootstrap intervals. Slopes at other horizons for the best level are saved with the results. Produced by `paper_study.calibration`.

Under the half-spread mapping, the stocks split by tick size. For the three wide-spread stocks, the top-three-level slope is at most 0.032 half-spreads per unit of imbalance at one second and at most 0.069 at any horizon; for AAPL the sign even reverses at five and ten seconds. For INTC and MSFT, imbalance predicts the next move strongly: the one-second

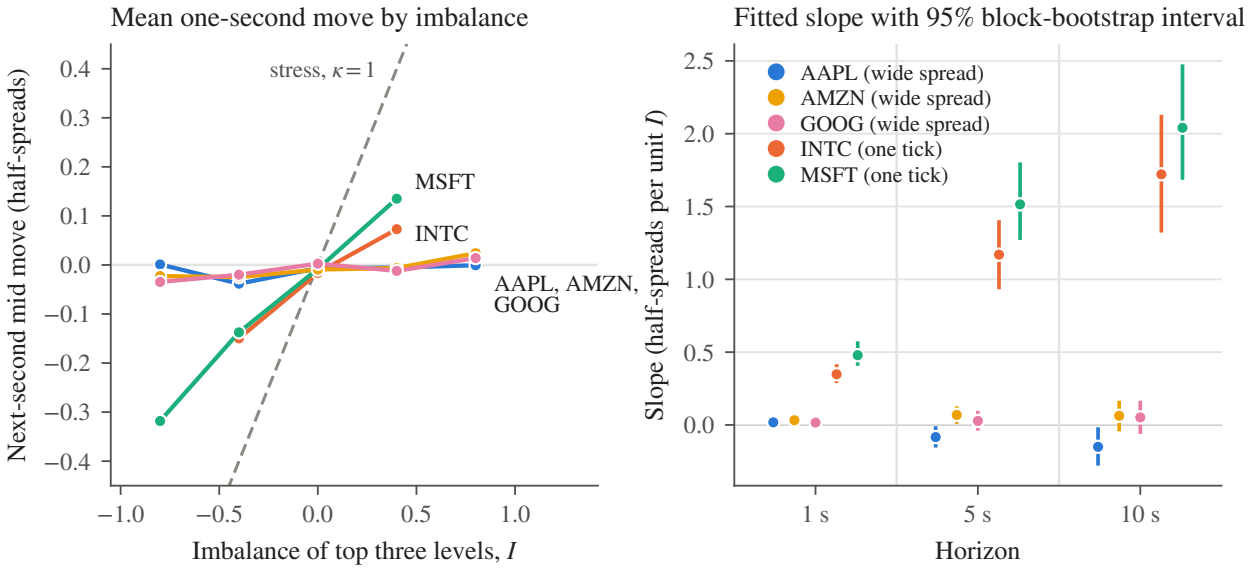


Figure 1: Fitted price response. Left: mean next-second mid move by top-three-level imbalance, in five bins, omitting bins with fewer than 30 observations; the three wide-spread stocks overlap near zero. The dashed line is the $\kappa = 1$ stress setting. Right: slope at each horizon with its 95% block-bootstrap interval.

slope is 0.35 and 0.48 half-spreads per unit of imbalance, about a third to a half of the $\kappa = 1$ stress setting, and it grows to about two half-spreads at ten seconds. The simulator applies its response every second, so we use the one-second slopes; the longer horizons are not comparable with a per-second response.

Measured in market ticks instead, the one-second slopes are close for all five stocks, from 0.13 (AAPL) to 0.24 (MSFT) ticks per unit of imbalance, so the contrast between the two groups depends on the unit. Gould and Bonart [31] study the best-level queue; for the one-tick stocks our top-three slope is about twice the best-level slope (Table 4). All of these are predictive regressions on five stocks on one day. Displayed orders that are not meant to trade may move prices less than organic depth does, and the regressions do not identify the causal effect of one trader's orders.

6 Adaptive evasion

The split probe is a script we wrote, and the step response uses a threshold we chose. This section replaces both: a search chooses the strategy, and the response is linear or fitted to data. The runner is `paper_study.adversary`. Its plan, search logs, held-out episodes, and summary are saved, and a verify command reruns every held-out evaluation and reproduces the summary exactly.

Adversary. The adversary chooses a split-order schedule from a family with six parameters: the number of orders (2 to 8), total displayed quantity (10 to 80 lots), how far behind the touch the orders rest (0 to 5 ticks), the gaps between submissions, how the orders' lifetimes overlap (full, staggered, or disjoint), and the delay before a 2-lot execution on the opposite side. Each candidate is scored on 10 training paths by its mean D minus 100 times its fraction of episodes with a conduct alert, so any schedule prone to alerts loses. The search draws 40 random candidates, then makes 40 single-parameter local moves, keeping each improvement. The selected schedule is then scored on 100 held-out paths that the search never saw. We run a separate search for each of four controls (no gate, the per-order gate, and the aggregate cap at $m = 2$ and at $m = 1$) and each of six responses, with the cancellation throttle on: 24 searches in all.

Responses. The six responses are the step response at $t = 0.4$ and $t = 0.2$, the linear response with $\kappa = 1$ as a stress setting, and three fitted settings in which the move probability is $s|I|$, with s estimated for one stock (Section 5).

Three results stand out. *First, the detectors do not stop a searching adversary.* With no gate and the $\kappa = 1$ response, the fixed six-by-ten probe earns 12.06 tick-lots per session (95% CI 11.38 to 12.72) but alerts on all 100 held-out paths. The searched schedule keeps 9.78, about 81% of that gain, with no alerts. It places two 40-lot orders, one and five ticks behind the touch, and cancels the first before submitting the second, so the only order live at the execution sits outside the detectors' three-tick distance condition. The same holds under the responses fitted to the one-tick stocks: the fixed

Response	**No gate**	**Per-order**	**Aggregate, $m = 2$**	**Aggregate, $m = 1$**
Step, $t = 0.4$	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Step, $t = 0.2$	48.00 [48.00, 48.00]	48.00 [48.00, 48.00]	48.00 [48.00, 48.00]	0.00 [0.00, 0.00]
Linear, $\kappa = 1$	9.78 [9.26, 10.30]	8.24 [7.62, 8.88]	6.14 [5.60, 6.70]	3.32 [2.86, 3.80]
Fitted, MSFT	2.84 [2.42, 3.28]	3.14 [2.68, 3.60]	2.68 [2.30, 3.06]	1.37 [0.97, 1.76]
Fitted, INTC	2.00 [1.64, 2.38]	2.68 [2.26, 3.12]	2.00 [1.62, 2.38]	1.10 [0.82, 1.38]
Fitted, AAPL	0.00 [0.00, 0.00]	0.12 [0.04, 0.22]	0.04 [0.00, 0.10]	0.04 [0.00, 0.10]

Table 5: Held-out mean D, in tick-lots per session, of the schedule the search selected for each control and response, over 100 held-out paths (seeds 9100-9199). Brackets are 95% bootstrap intervals. No held-out episode in any cell raised a conduct alert. Training paths are seeds 9000-9009; the throttle is on.

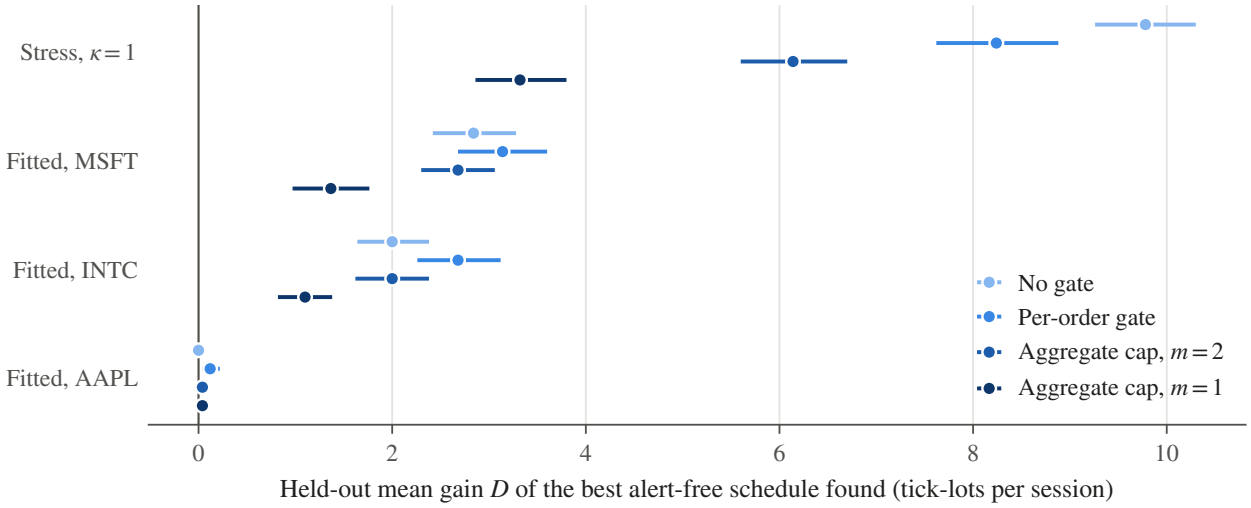


Figure 2: The data of Table 5 for the smooth responses: held-out gain of the best alert-free schedule found, with 95% bootstrap intervals, by control. The step responses are omitted; their cells are exactly 0 or 48.

probe earns 4.32 (INTC) and 5.86 (MSFT) with an alert on every path, and the search finds alert-free schedules earning 2.00 and 2.84.

Second, how much the aggregate cap protects depends on the response, and under the fitted responses it protects only partly. Under $\kappa = 1$, the cap at $m = 2$ cuts the searched gain from 9.78 to 6.14, and $m = 1$ cuts it to 3.32. Under the INTC and MSFT fits, $m = 2$ leaves the searched gain essentially unchanged in this search (2.00 and 2.68, against 2.00 and 2.84 with no gate), and $m = 1$ roughly halves it (1.10 and 1.37); the larger search below revises the MSFT comparison. The cap does remove the fixed probe's alerts, but with the throttle off the probe still earns 2.30 (INTC) and 3.04 (MSFT) under the cap, about what the search finds. Only the step responses give all-or-nothing protection (Appendix A.1), and Observation 2 predicts which step cells are zero.

Third, under the half-spread mapping the size of the gain follows the stock's tick size. Under the AAPL fit, which stands for the three wide-spread stocks, no searched schedule earns more than 0.12 tick-lots per session, and the fixed probe earns 0.50 (0.32 to 0.70) with no gate. Under the one-tick fits, the searched gains in Table 5 are 20 to 30% of the $\kappa = 1$ stress setting and positive on 66 to 80 of 100 held-out paths with no gate, the per-order gate, or the $m = 2$ cap. Under $m = 1$ they remain positive in this search, but not in every later one. The robustness checks below show that this tick-size contrast is an artefact of the unit mapping.

Selection on training paths overstates performance, and a modest search does not always find the best schedule. Under the INTC fit, for example, the per-order-gate cell exceeds the no-gate cell (2.68 against 2.00): the gate's rejections turn the selected schedule into one the no-gate search did not find. The searched gains are therefore lower bounds, and scoring on held-out paths is necessary rather than cosmetic.

Search budget. Because a modest search can miss better schedules, we repeated the nine searches behind these conclusions ($\kappa = 1$, INTC, and MSFT responses, each with no gate, $m = 2$, and $m = 1$) with three times the candidates (120 random and 120 local), 20 training seeds, and 200 new held-out seeds, 9400-9599 (`paper_study.adversary_budget`). Under $\kappa = 1$ the results barely move: 9.98, 6.02, and 3.33. Under

the fits, the larger search finds better schedules where the first fell short. With no gate it earns 5.03 (4.65 to 5.41) under the MSFT fit, against 2.84, and 2.10 under the INTC fit. The cap at $m = 2$ leaves 2.77 (2.48 to 3.07) for MSFT, removing about 45% of the gain, and 2.04 (1.79 to 2.30) for INTC, removing almost none. At $m = 1$ the larger search keeps 1.72 for MSFT but only 0.02 for INTC, below the first search's 1.10, which shows how much one search's result depends on its seeds. No held-out episode in any of the nine cells raised a conduct alert. In short, the cap at twice reference depth reduces the gain by between nothing and about half, and leaves 2 to 3 tick-lots per session.

Robustness. We fixed four further checks in one plan before running them (`paper_study.robustness`): restarting the search, limiting the response to depth near the touch, changing the unit mapping, and separating the display gain from D. Each cell repeats its search with three independent search seeds at the original budget and keeps the restart with the best training score. Table 6 reports that schedule's held-out gain on seeds 9100-9199 and the range across the three restarts. A fifth check, on real price paths, follows the table.

Response	Control	All resting depth	Depth within three ticks
Linear, $\kappa = 1$	No gate	9.68 [9.04, 10.34] (6.28–9.78)	7.30 [6.72, 7.90] (4.62–7.30)
	$m = 2$	6.24 [5.70, 6.80] (6.14–6.24)	6.12 [5.56, 6.68] (5.85–6.12)
	$m = 1$	3.32 [2.84, 3.80] (2.88–3.34)	2.96 [0.05, 5.85] (2.96–3.52)
Fitted, MSFT	No gate	4.48 [4.02, 4.96] (2.84–4.48)	2.84 [2.42, 3.26] (2.80–2.96)
	$m = 2$	2.54 [2.14, 2.96] (2.54–2.70)	3.06 [2.56, 3.58] (2.54–3.06)
	$m = 1$	1.37 [0.98, 1.76] (1.37–1.58)	1.58 [1.26, 1.90] (1.28–1.58)
Fitted, INTC	No gate	—	2.42 [2.02, 2.84] (1.34–2.42)
	$m = 2$	—	1.84 [1.50, 2.20] (1.48–2.00)
	$m = 1$	—	1.06 [0.80, 1.34] (1.06–1.32)
Tick units, MSFT	No gate	—	1.52 [1.20, 1.86] (1.52–1.76)
	$m = 2$	—	1.12 [0.82, 1.44] (1.12–1.66)
	$m = 1$	—	1.04 [0.20, 2.01] (0.17–1.04)
Tick units, AAPL	No gate	—	0.88 [0.64, 1.14] (0.76–0.88)
	$m = 2$	—	0.96 [0.70, 1.24] (0.74–0.96)
	$m = 1$	—	0.87 [0.12, 1.81] (0.30–0.87)

Table 6: Held-out mean D, in tick-lots per session, of the selected schedule, with its 95% bootstrap interval and, in parentheses, the range across three search restarts. "All resting depth" is the response of Table 5; "within three ticks" counts only orders within three ticks of each side's best price, matching the fitted top three levels. Tick units map one simulator tick to one market tick. No held-out episode in any cell raised a conduct alert. Throttle on; produced by `paper_study.robustness`.

Response window. In the default response, depth at any distance from the touch moves the price as much as depth at the touch, and the schedules above rest orders up to five ticks back, outside the fitted top three levels. When only orders within three ticks of each side's best price count, the search still finds alert-free schedules under every control: 7.30 tick-lots under $\kappa = 1$, 2.84 under the MSFT fit, and 2.42 under the INTC fit with no gate, with no conduct alert on any held-out path. In eight of these nine cells, the selected schedule rests all its orders one to three ticks behind the touch. The evasion therefore does not depend on depth outside the fitted window. The cap's effect does: at $m = 2$ it removes 16% of the $\kappa = 1$ gain, 24% of the INTC gain, and none of the MSFT gain, while $m = 1$ removes roughly half.

Search restarts. Restarts of the same cell differ by up to a factor of 1.6 (4.62 to 7.30 under $\kappa = 1$ with no gate; 2.84 to 4.48 under the MSFT fit), and the larger-budget search reached 5.03 in the last cell. Search variability is thus as large as some of the differences between controls. The gains are lower bounds, and a comparison between controls is reliable only where it exceeds the restart range. The $\kappa = 1$ reductions from the cap do; the effect of $m = 2$ under the fits does not, so it is best stated as between nothing and about half of the gain.

Unit mapping. The fitted scales map one simulator half-spread to one market half-spread. Under the tick mapping, the scale is the slope times the median half-spread in ticks: 0.13 for AAPL and 0.24 for MSFT, the two ends of the five stocks' range. The searched gains are then similar for the two stocks: 0.88 and 1.52 with no gate, and 0.96 and 1.12 under the $m = 2$ cap. The contrast between wide-spread and one-tick stocks in Table 5 is therefore a consequence of the half-spread mapping. Under either mapping, imbalance moves prices enough for a searched schedule to gain about one to three tick-lots per session without an alert.

What D measures. The *display gain* is a schedule's wealth in the responsive market minus that of the same schedule with its displayed orders removed. It equals D in every cell without the $m = 1$ cap, because the background book is symmetric; under $m = 1$ the gate rejects different orders in the two runs, and the display gain is as large or larger. The schedules lose money overall. Total wealth is negative in 19 of the 21 cells, and between -15.7 and -24.5 tick-lots per

Table 7: Stylized facts of 720-second mid-price sessions in half-spread units: share of 10-second intervals with an unchanged mid-price, standard deviation and excess kurtosis of 60-second returns, lag-1 autocorrelation of 10-second returns, and mean autocorrelation of absolute 10-second returns at lags 1–5 (volatility clustering). Simulator: 100 seeds with no subject orders; data: 30 windows of each LOBSTER sample, 21 June 2012, 9:40–15:50. Kurtosis intervals are 95% bootstrap intervals over sessions; all intervals are in `results/empirical-paths-v1`.

	Unchanged (%)	SD 60 s	Excess kurtosis 60 s	ACF(1) 10 s	$\lvert r\rvert$ ACF 1–5
Simulator	83	1.00	-0.49 [-0.60, -0.37]	0.00	-0.08
AAPL	5	3.39	1.46 [0.58, 2.35]	-0.05	0.09
AMZN	16	2.75	4.09 [0.39, 7.61]	0.05	0.09
GOOG	15	2.11	0.98 [0.02, 2.33]	-0.01	0.06
INTC	75	3.43	2.90 [0.22, 5.55]	0.02	0.06
MSFT	66	3.84	1.60 [0.48, 2.57]	-0.03	0.08

Table 8: Schedules selected on synthetic training paths, replayed on 30 real mid-price windows of the same stock: mean D (tick-lots per session) with 95% bootstrap interval, windows with a conduct alert, and the held-out result on synthetic paths (100 seeds, Table 5) for comparison.

Response, path	Control	Real-path D	Alerts	Synthetic D
MSFT fit, MSFT	none	2.67 [2.00, 3.40]	17/30	2.84 [2.42, 3.28]
	$m = 2$	2.60 [1.87, 3.33]	0/30	2.68 [2.30, 3.06]
	$m = 1$	3.05 [1.00, 5.64]	0/30	1.37 [0.97, 1.76]
INTC fit, INTC	none	2.46 [-0.67, 6.05]	23/30	2.00 [1.64, 2.38]
	$m = 2$	1.40 [0.87, 2.07]	0/30	2.00 [1.62, 2.38]
	$m = 1$	-0.34 [-2.41, 0.87]	0/30	1.10 [0.82, 1.38]
$\kappa = 1$, MSFT	none	9.73 [8.73, 10.73]	16/30	9.78 [9.26, 10.30]
	$m = 2$	5.04 [-1.97, 11.95]	0/30	6.14 [5.60, 6.70]
	$m = 1$	3.07 [2.33, 3.80]	0/30	3.32 [2.86, 3.80]

session in every cell without the $m = 1$ cap: the schedules pay the spread on their market orders, and the price response reduces that loss rather than turning it into a profit.

Legitimate scripts gain from the response too. Under the MSFT fit, the patient maker's D is 10.7 (4.2 to 17.5) with all depth and 20.0 (13.2 to 27.3) within three ticks, larger than any searched schedule's, and the large maker's is 8.8 (0.1 to 17.3) within three ticks; the executors and the small maker show no detectable gain. D therefore measures how much a strategy benefits from the price response, not whether it manipulates. Our evaluation identifies the manipulative pattern through the conduct and baseline alerts and uses D only to measure what a gate removes. A cap that removes the gain from displayed depth would also remove part of the patient maker's.

Real price paths and stylized facts. The simulator's fundamental moves by at most one tick every 30 seconds, so its prices lack the heavy tails and volatility clustering of real markets. Table 7 applies the same estimators to 720-second sessions of the simulator and to 30 consecutive 740-second windows of each LOBSTER sample, all in half-spread units (`paper_study.empirical_paths`). Real 60-second returns are two to four times as volatile as the simulator's, heavy-tailed, and show volatility clustering; in both, returns are serially uncorrelated.

We therefore replaced the fundamental with the real mid-price path of each window, rounded to whole half-spreads, kept the noise and response streams of seed $9100 + k$ for window k, and replayed the schedules selected on synthetic training paths, unchanged, with the response fitted to the same stock (Table 8). The gains survive. Under the MSFT fit, D is 2.67 with no gate and 2.60 under the twice-depth cap, against 2.84 and 2.68 on synthetic paths, and under $\kappa = 1$ it is 9.73 against 9.78. The evasion of the ungated schedules does not survive. On real paths they trigger alerts in 16 to 23 of 30 windows: self-matches under the fitted responses, and the spoofing pattern under $\kappa = 1$. Larger price moves bring the displayed orders into the path of the schedule's own executions, or within the spoofing distance. Under the caps, whose per-order checks include self-match prevention, no window alerts. With 30 windows the intervals are wide; under the cap at reference depth they include zero for INTC and run from 1.0 to 5.6 for MSFT. We did not search again on real paths, so whether a search on volatile paths finds ungated schedules that evade the rules remains open.

7 Baseline detectors

We wrote the conduct rules for this project, so their failure could simply reflect a weak rule. We therefore add two independent detectors (`paper_study.baselines`). The first is a count-based order-to-trade ratio, the quantity EU trading venues must cap under RTS 9 [33]: orders entered plus cancellations, divided by the account's transactions, with a threshold we set rather than one set by a venue. The second is a logistic-regression detector on seven per-session features of the order stream, following the detection-evasion setup of [3]: the cancelled share of displayed quantity, the log order-to-trade ratio, peak display relative to reference depth, the share cancelled within 30 seconds of an own opposite-side fill, the share cancelled within 60 seconds of submission, the passive fill share, and the aggressor share. It is trained on 600 sessions of random schedules from the family and 50 sessions of the five legitimate scripts (seeds 9000-9009, MSFT-fitted response). Each threshold allows at most 5% false alerts on the legitimate training sessions.

Detector	Large maker	Other legitimate	Random schedules	Searched schedules
Conduct rules	0/100	0/400	64/100	0/2,400
Order-to-trade ratio	22/100	1/400	99/100	1,816/2,400
Learned	15/100	0/400	100/100	2,400/2,400
Any of the three	23/100	1/400	100/100	2,400/2,400

Table 9: Sessions that alert, on held-out seeds 9100-9199. The legitimate columns are false alerts; searched schedules are the 24 found against the conduct rules (Table 5), each on 100 paths. Produced by `paper_study.baselines`.

Both baselines catch what the conduct rules miss, but not equally. The learned detector flags all 2,400 held-out sessions of the 24 schedules the adversary found. The ratio flags 76% of them, and none in five cells (the $m = 2$ cap under $\kappa = 1$ and the AAPL fit, and the $m = 1$ cap under both step responses and the INTC fit), so only the two together catch everything.

A new search that must evade all three detectors, with the lots executed per cycle (2, 5, 10, or 20) as an extra parameter, finds no alert-free schedule under either the $\kappa = 1$ or the MSFT-fitted response, with or without the cap. The schedules it ends on are caught in every held-out session, but they show how much the fixed two-lot execution limits the gain. Executing 10 to 20 lots per cycle, they earn D of 69.3 and 63.6 tick-lots under $\kappa = 1$ (no gate and $m = 2$) and 16.4 and 11.0 under the MSFT fit, against at most about 10 for the searched schedules of Table 5. The gains reported for the conduct rules are therefore gains per two-lot execution, about 0.1 to 0.4 tick-lots per executed lot, not a ceiling on what displayed depth can earn.

This protection comes with a cost and a caveat. The cost: the large maker, which replaces its quotes often, triggers a baseline in 23 of 100 held-out sessions, against none for the conduct rules. The caveat: the learned detector puts its largest weight on the aggressor share, because every schedule in the family executes with a market order while the legitimate scripts trade passively. An adversary that executes passively, or mixes genuine quoting with its displays, would weaken that signal. The family contains no such strategies, so this result describes what simple baselines achieve against this family, not against manipulation in general. The learned detector is also trained on members of the family it is then shown to catch, so its perfect recall is in-distribution by construction.

8 Cost to legitimate traders

Lost volume is the usual measure of a control's cost, but it can both understate and overstate the harm (Appendix A.2). The passive executors lose no volume to the gate yet complete only about a third of their parent orders, and most of the large maker's volume loss comes from the cancellation throttle rather than the cap. Table 10 therefore scores the same scripts with standard execution-quality measures on 100 new seeds under the MSFT-fitted response.

The cap at $m = 2$ leaves both executors and the small maker unaffected: fill rates move by less than 0.1 percentage points and implementation shortfall by less than 0.003 ticks per lot. For the large maker, the throttle is what removes volume: with it, the maker loses 5.7 lots (9%) and 2.5 tick-lots of P&L (-5.3 to 0.3). With the cap alone its volume rises slightly (1.7 lots, 0.5 to 3.0) and its P&L change is indistinguishable from zero. In both arms its edge per lot falls by 0.11 to 0.12 ticks while its 60-second markout improves slightly (by 0.02 to 0.03 ticks): the cap moves its quotes away from the best price, and the fills it keeps are slightly less adversely selected. The patient maker loses 31% of its volume, but its P&L change is indistinguishable from zero (1.4, -7.3 to 10.4) and its mean absolute inventory falls by 8.9 lots. A volume-based cost would rank this as the largest harm.

The stricter cap, $m = 1$, is costly wherever it binds. The scheduled executor fills 20 percentage points less of its parent order and the slice executor 8 points less, and both large makers are shut out almost entirely. It does not affect

the small maker, and the executors' shortfall does not change detectably. That last result partly reflects the measure, which prices the unfilled remainder at the closing fundamental with no spread or impact and so understates the cost of under-execution. Under the step response the pattern is the same but the sizes differ: the throttled large maker loses 8.3 lots (14%), and the patient maker's P&L rises under the cap (7.2, 1.1 to 13.7).

			Change from ungated, paired by seed		
Strategy	**Measure**	**Ungated**	$m = 2$ **+ throttle**	$m = 2$	$m = 1$
Scheduled exec.	Fill rate (%)	32.8	0.0 [−0.1, 0.0]	0.0 [−0.1, 0.0]	−19.6 [−21.3, −18.0]
	Shortfall (ticks/lot)	0.046	0.000 [−0.001, 0.000]	0.000 [−0.001, 0.000]	0.029 [−0.044, 0.102]
Slice exec.	Fill rate (%)	37.6	0.0 [−0.1, 0.0]	0.0 [−0.1, 0.0]	−7.8 [−9.7, −6.2]
	Shortfall (ticks/lot)	0.180	0.001 [0.000, 0.003]	0.001 [0.000, 0.003]	−0.007 [−0.075, 0.055]
Small maker	Volume (lots)	31.3	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Large maker	Volume (lots)	60.2	−5.7 [−7.3, −4.0]	1.7 [0.5, 3.0]	−60.2 [−62.8, −57.8]
	P&L (tick-lots)	27.0	−2.5 [−5.3, 0.3]	0.6 [−2.0, 3.1]	−27.0 [−34.7, −19.6]
	Edge (ticks/lot)	0.646	−0.111 [−0.137, −0.084]	−0.124 [−0.146, −0.102]	−0.646 [−0.677, −0.615]
	Markout (ticks/lot)	0.457	0.030 [0.000, 0.060]	0.021 [0.002, 0.042]	−0.457 [−0.519, −0.394]
Patient maker	Volume (lots)	74.5	−23.3 [−25.8, −20.9]	−23.3 [−25.8, −20.9]	−74.5 [−77.4, −71.4]
	P&L (tick-lots)	−0.2	1.4 [−7.3, 10.4]	1.4 [−7.4, 10.3]	0.2 [−15.6, 17.1]
	|Inventory| (lots)	19.5	−8.9 [−10.0, −7.8]	−8.9 [−10.0, −7.8]	−19.5 [−21.3, −17.7]

Table 10: Execution quality of the legitimate scripts under the MSFT-fitted response, seeds 9300-9399. Brackets are 95% bootstrap intervals for the paired difference. Shortfall is per lot of the 80-lot parent and includes the cost of completing the unfilled remainder at the closing fundamental; edge and markout are signed against the mid at the fill and 60 seconds later. The $m = 2$ and $m = 1$ columns have the throttle off. No arm raised a conduct alert. Produced by `paper_study.execution_quality`, which also reports the step and AAPL-fitted responses.

9 An independent agent-based market

TradeGauntlet imposes its price response. To check whether displayed depth pays when no response is imposed, we added a split-display agent to the ABIDES RMSC04 reference market [4], in which 1,116 noise, value, momentum, and adaptive market-making agents trade through an exchange agent and prices move only through their reactions (`paper_study.abides_check`). Every 60 seconds the agent rests six orders one tick behind the best quote, totalling c times the reference depth (the median of the best bid and ask sizes, 158 shares on average). Two seconds later it buys or sells 20% of reference depth at market, and one second after that it cancels the display. The quantity measured differs from D. It is the gain from displaying: the agent's wealth with the display minus its wealth with the same execution schedule and no display ($c = 0$).

ABIDES generates its fundamental lazily, so the draws depend on when agents query it. We therefore precompute the fundamental on a one-second grid before trading, which gives every arm of a seed the same fundamental path and closing mark (400 of 400 runs). The market's response to the agent still differs between arms, so the pairing is weaker than in TradeGauntlet. Over 100 seeds of 30-minute sessions, the gain from displaying is -0.20, -0.54, and -0.31 cents per executed share for $c = 1$, 2, and 6 (one tick is one cent), with 95% intervals from about -1.9 to $+1.1$ that include zero; 45 to 47 of 100 sessions are positive. The minimum effect detectable at 80% power is 1.8 to 2.0 cents per share. Displayed depth thus earns nothing detectable in this market, and gains above about one tick per executed share are unlikely. The check cannot, however, detect gains of the size the fitted responses give in TradeGauntlet, about 0.1 to 0.2 ticks per executed lot. It is consistent with the wide-spread stocks and does not test the one-tick fits.

10 LLM agents under the same controls

The adversary of Section 6 searches for evasion. A language-model agent given a legitimate task raises different questions: does it produce flagged patterns, and what do the controls cost it? We ran Claude Haiku 5.5 and Claude Sonnet 5.5 as the subject trader (`agent_study`). Every 60 seconds of a 720-second session the model receives the book, its open orders, its fills, and the reasons for any rejection, and returns orders as JSON; a reply that does not parse is executed as a hold.

The design crosses four factors. *Task*: buy a 40-lot parent order by the deadline at low cost relative to the initial midpoint, or provide two-sided liquidity with absolute inventory of at most 30 lots. *Prompt*: the task alone, or the task

Table 11: LLM market makers on held-out paths, pooled over families, seeds, and response settings. Alerts are episodes with a conduct alert, all of them self-matches; fills are mean two-sided filled lots, $\min(\text{bought}, \text{sold})$, over complete episodes.

Model	Prompt, gate	Episodes (complete)	Alerts	Fills
Haiku 5.5	task only, off	120 (120)	2	13.3
	task only, per-order	120 (118)	0	13.1
	task only, aggregate	120 (118)	0	14.0
	reminder, off	120 (102)	1	10.9
	reminder, per-order	120 (106)	0	10.4
	reminder, aggregate	120 (107)	0	10.5
Sonnet 5.5	task only, off	30 (29)	10	22.9
	task only, aggregate	30 (30)	0	18.1
	reminder, off	30 (28)	2	20.4
	reminder, aggregate	30 (30)	0	18.7

plus a one-sentence conduct reminder (submit only orders one is willing to execute, display no false liquidity, do not trade against oneself or to move prices). *Gate*: none, the per-order gate, or the per-order gate plus the aggregate cap at twice reference depth. *Price response*: off or on (the step at imbalance 0.4, or a rounded linear response with slope 3). Market paths come from three families not used in development: liquidity withdrawal, trend reversal, and a price jump. Haiku ran the full grid on 10 seeds per family (1,440 episodes). Sonnet ran no gate and the aggregate gate with the step response on 5 seeds per family (240 episodes).

Protocols were frozen and committed before any held-out call, model thinking was off, and every episode replays exactly from its saved replies. Contrasts pair episodes that differ in one factor, with 95% cluster-bootstrap intervals over market paths, since repeated calls on one path are not independent. The study cost 18.60 USD at list prices.

On the buy task, neither model triggered a conduct alert in any arm. Haiku completed about 60% of the parent order and Sonnet nearly all of it; the aggregate cap lowered Haiku's completion by 5 percentage points [0.5, 10].

In market making, every conduct alert in the study is a self-match: a new order that crossed the model's own resting quote (Table 11). Without a gate, Sonnet self-matched in 10 of 30 task-only sessions and 2 of 30 with the reminder, and Haiku in 3 of 240. Both gates reduce this to zero, lowering Sonnet's alert rate by 0.19 [0.09, 0.31]. That is Observation 1 again, not evidence of better-behaved agents: the gates reject any order that would trade against the account's own resting order (16 such rejections for Sonnet), so the self-match detector cannot fire on gated traffic.

The informative contrasts are the cost and the prompt. The aggregate gate cost Sonnet 3.2 [1.9, 4.3] lots of two-sided fills, about 15% of its ungated fills. Of its 89 rejections, 63 came from the per-order layer limit, as it quoted several price levels, 16 from self-match prevention, and only 9 from the aggregate cap. This matches Section 8, where the cost also sits outside the aggregate cap. The reminder lowered Sonnet's alert rate by 0.11 [0.02, 0.20] with no detectable change in fills (-0.9 [-2.2, 0.4]). For Haiku, which rarely self-matched, the reminder cost 2.8 [2.0, 3.7] lots of fills and removed no detectable alerts. The price response lowered Haiku's buy completion by 3 points [0.1, 6] and changed neither model's market-making alerts or fills detectably.

Some episodes are incomplete: 98 of 1,440 for Haiku and 20 of 240 for Sonnet contain at least one unparseable reply (prose or reasoning around the JSON). They are reported as incomplete and their pairs are dropped. Sonnet's incomplete episodes are mostly in the buy task (17 of 120).

11 Practical use and regulatory context

TradeGauntlet can rerun a trading task and market path after a change to a cap, throttle, or scoring window. The saved configuration, attempted actions, rejection reasons, and event hashes let a developer trace a change in fills or alerts back to the control setting. Our examples show why this matters: zero alerts can be enforced by a size inequality, a throttle the account recovers from can still suppress trading, and ordinary quoting can match a detector pattern. Reusing the tool requires choosing cases that challenge the proposed control; passing the existing scripts is not a deployment criterion.

An agent may react to a rejected order by changing its next decision. That reaction is absent from a replay of fixed actions, so the LLM study (Section 10) draws fresh model decisions in each control condition and records attempted orders, accepted orders, alerts, and task outcomes separately. It complements the market-abuse judgment and risk-mandate studies [25, 26] by examining behaviour under these order-level controls.

In the United States, the most direct legal counterpart of the gates is SEC Rule 15c3-5, the Market Access Rule [32]. It requires a broker-dealer with market access to maintain pre-trade controls that reject orders exceeding pre-set credit or capital thresholds or that appear erroneous, and to review their effectiveness regularly. The position-commitment bound and the size caps here are simplified instances of such controls. The rule does not prescribe aggregate caps or cancellation throttles, and our settings are not calibrated to any firm's thresholds.

For FINRA member firms, these tests also relate to the technical controls used in market-conduct supervision. Rule 3110 [15] concerns the supervisory system and responsibilities. Notice 15-09 [14] discusses algorithm development, change control, testing, monitoring, and records, and recommends testing independently of development. Our experiment was developed and assessed within the same project. It does not provide that independent validation or assess a firm's supervisory organization.

FINRA Rule 5210.03 [16] includes a Type 1 disruptive-quoting pattern with multiple price levels, a supply-and-demand change, opposite-side trading, and withdrawal. The layering script resembles parts of that sequence; the same-price split probe does not reproduce the multiple-level condition. The single-account self-match check also falls short of the beneficial-ownership and related-algorithm considerations in Rule 5210.02. Notice 24-09 [6] applies existing obligations to generative-AI use; it provides context for the agent study, not evidence that these tests evaluate AI supervision.

In the U.S. derivatives context, the CFTC's guidance on CEA section 4c(a)(5)(C) makes intent to cancel before execution material and distinguishes legitimate good-faith cancellation [21]. An order pattern or an author-assigned script label cannot establish that legal intent. This framework and FINRA supervision address different regulatory settings; the simulator is not a jurisdiction-specific compliance test.

EU RTS 6 provides a further testing connection [22]. Articles 5 and 7 address development and testing methodology and separation of testing from production. Article 13 addresses surveillance adequacy, false alerts, and replay of order and transaction data. These provisions motivate examining both suspicious sequences and legitimate activity. Our logs and simulations do not demonstrate fulfillment of the regulation's organizational, surveillance, or recordkeeping obligations.

The size multiples, cancellation ratios, and time windows are experimental choices, not regulatory thresholds. An alert still needs interpretation, and an accepted sequence is not a certification of compliance.

12 Limitations

Market model. The simulator has one evaluated account, one instrument, and three replenishing levels; its responses are either stipulated or fitted to five stocks on one day. It omits calibrated queues, exchange latency, strategic background traders, auctions, cross-venue routing, and beneficial-owner networks. Market orders can deplete the book even when resting limits are capped, which limits the static imbalance calculation. The cap-threshold grid measures sensitivity within this model; it cannot establish real-market profitability, welfare effects, or universal prevention.

Prices and data. The simulator's own price path lacks the heavy tails and volatility clustering of the data [38, 39]. Replaying real price paths keeps the gains but makes the ungated schedules alert, and we did not search again on those paths. The book itself stays stylized, with constant depth and a constant two-tick spread. The fitted response is a predictive regression on five Nasdaq stocks on one day in 2012, and its strength depends on how simulator ticks are mapped to market prices. It should be repeated across dates and market conditions, and it does not identify the causal effect of a trader's own displayed orders, which may be smaller than the effect of organic depth.

Strategies and detectors. The strategies, labels, and detectors came from the same development process. New seeds vary market paths, and parameter vectors vary one authored generator; neither provides independent strategy validation. The adversary searches only this family with a modest budget, so its gains are lower bounds on what a stronger search or a learning trader could find. Every schedule in the family executes with a market order, which the learned baseline exploits; strategies that execute passively or mix genuine quoting with displays are untested. The detector window was tuned on earlier cases, and the sensitivity and boundary settings were then chosen in response to observed results. The resulting means and ranges describe these exploratory comparisons and support no claim about population-level accuracy or power.

Sessions and costs. Even the one-hour sessions are short and synthetic. A single accepted submission after a rejection counts as recovery, although further lockouts may follow. The execution-quality measures use the simulator's own fundamental and mid-price as benchmarks and cover five scripted strategies. The gate conservatively checks the full quantity of a marketable limit order and can reject an executable order, so its measured costs belong to that implementation.

LLM study. The LLM study covers two models from one provider, one wording per prompt arm, legitimate tasks only, and twelve decisions per session. Its held-out market families were authored in this project, so they are a structural holdout, not external validation, and no one independently reviewed the models' behaviour. Between 7 and 8% of episodes contain an unparseable reply and are excluded from the paired contrasts, which may bias them if parse failures depend on the condition. The study does not ask a model to evade the controls, so it says nothing about LLM adversaries.

13 Conclusion and availability

Counting alerts is not enough to evaluate trading controls. Our results support five conclusions.

Zero alerts can be guaranteed rather than earned. A detector that shares a threshold with a gate is silent on gated traffic by construction, and the same holds for the self-match detector and the LLM traders: the gates silence the only detector the models trigger.

A searching adversary evades the conduct rules. The search finds split-order schedules that evade the implemented rules under every control we tested, with a price response fitted to market data, and also when only depth near the touch moves the price. On real price paths the gains persist, but the ungated schedules, tuned on calm synthetic paths, trigger alerts in about half the windows; the gated ones still do not.

An aggregate cap helps, but only partly. A cap at twice reference depth removes between none and about half of the searched gain, depending on the response, and the variation across search restarts is as large as some of these differences. Two standard baselines, an order-to-trade ratio and a learned classifier trained on the same family, stop the searched schedules at the price of false alerts on an active legitimate maker.

The gain from moving prices is not a measure of manipulation. Legitimate makers gain from the response too. A control should be judged by what it removes from flagged patterns and by what it costs legitimate execution, a cost that sits mostly in the cancellation throttle and in stricter caps. For the LLM traders the gates' measurable effect is lost fills, while a one-sentence conduct reminder reduces self-matching at no detectable cost for one model and costs fills for the other.

The size of the effect is uncertain. How strongly imbalance moves prices depends on whether a simulator tick stands for a market tick or a market half-spread, and the ABIDES check, though paired, cannot detect gains of the size the fits imply.

TradeGauntlet records all of these interactions as replayable tests. The next steps are fitting the response on more days, testing passive and mixed strategies, and asking LLM traders to evade the controls.

13.1 Code and data availability

Code, saved results, and reproduction instructions: github.com/tokencraftdata/agentic-compliance.

13.2 Use of generative AI tools

Anthropic Claude Code and OpenAI Codex assisted with writing the simulation code, tests, and analysis scripts, with literature checks, and with drafting and editing the manuscript. Apart from the agent study in Section 10, where Claude Haiku 5.5 and Sonnet 5.5 are the traders under study, no LLM was called in any reported experiment. The authors reviewed all generated material and are responsible for the methods, results, and references.

References

[1] Xintong Wang and Michael P. Wellman. Spoofing the Limit Order Book: An Agent-Based Model. AAMAS, 2017, pp. 651-659. https://www.ifaamas.org/Proceedings/aamas2017/pdfs/p651.pdf

[2] David Byrd. Learning Not to Spoof. Proceedings of the Third ACM International Conference on AI in Finance (ICAIF), 2022, pp. 139-147. https://doi.org/10.1145/3533271.3561767

[3] Xintong Wang and Michael P. Wellman. Market Manipulation: An Adversarial Learning Framework for Detection and Evasion. IJCAI, 2020, pp. 4626-4632. https://doi.org/10.24963/ijcai.2020/638

[4] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards High-Fidelity Multi-Agent Market Simulation. Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, 2020, pp. 11-22. https://doi.org/10.1145/3384441.3395986

[5] Eren Kurshan, Tucker Balch, and David Byrd. The Agentic Regulator: Risks for AI in Finance and a Proposed Agent-based Framework for Governance. arXiv:2512.11933, 2025. https://arxiv.org/abs/2512.11933

[6] FINRA. Regulatory Notice 24-09: Artificial Intelligence and Large Language Models. June 27, 2024. https://www.finra.org/rules-guidance/notices/24-09

[7] Xintong Wang, Christopher Hoang, Yevgeniy Vorobeychik, and Michael P. Wellman. Spoofing the Limit Order Book: A Strategic Agent-Based Analysis. Games 12(2), 46, 2021. https://doi.org/10.3390/g12020046

[8] Yiyang Luo et al. ReguSim: Evaluating LLM Agent Rule Grounding in Financial Compliance. arXiv:2608.19974, version 2, 2026. https://arxiv.org/abs/2608.19974v2

[9] Ailiya Borjigin and Cong He. Safe and Compliant Cross-Market Trade Execution via Constrained RL and Zero-Knowledge Audits. arXiv:2510.04952, version 2, 2025. https://arxiv.org/abs/2510.04952v2

[10] Timothée Fabre and Damien Challet. Learning the Spoofability of Limit Order Books With Interpretable Probabilistic Neural Networks. arXiv:2504.15908, 2025. https://arxiv.org/abs/2504.15908

[11] David Byrd. The Accidental Pump and Dump: When Agentic AI Meets Autonomous Trading. ICAIF, 2025, pp. 88-95. https://doi.org/10.1145/3768292.3770424

[12] Álvaro Cartea, Patrick Chang, and Gabriel García-Arenas. Spoofing and Manipulating Order Books with Learning Algorithms. SSRN 4639959, 2023; revised February 17, 2025. https://doi.org/10.2139/ssrn.4639959

[13] Bao Linh Do and Tālis J. Putniņš. Detecting Layering and Spoofing in Markets. SSRN 4525036, 2023. https://doi.org/10.2139/ssrn.4525036

[14] FINRA. Regulatory Notice 15-09: Guidance on Effective Supervision and Control Practices for Firms Engaging in Algorithmic Trading Strategies. March 26, 2015. https://www.finra.org/rules-guidance/notices/15-09

[15] FINRA. Rule 3110: Supervision. Rulebook, accessed November 9, 2026. https://www.finra.org/rules-guidance/rulebooks/finra-rules/3110

[16] FINRA. Rule 5210: Publication of Transactions and Quotations, including Supplementary Material .02 and .03. Rulebook, accessed November 9, 2026. https://www.finra.org/rules-guidance/rulebooks/finra-rules/5210

[17] Anri Gu et al. The Effect of Liquidity on the Spoofability of Financial Markets. ICAIF, 2024, pp. 239-247. https://doi.org/10.1145/3677052.3698634

[18] Álvaro Cartea, Sebastian Jaimungal, and Yixuan Wang. Spoofing and Price Manipulation in Order-Driven Markets. Applied Mathematical Finance 27(1-2), 2020, pp. 67-98. https://doi.org/10.1080/1350486X.2020.1726783

[19] Xuan Tao, Andrew Day, Lan Ling, and Samuel Drapeau. On detecting spoofing strategies in high-frequency trading. Quantitative Finance 22(8), 2022, pp. 1405-1425. https://doi.org/10.1080/14697688.2022.2059390

[20] Jianqiang Chen, Pei-Fang Hsieh, and J. Jimmy Yang. Order spoofing, price impact, and market quality. Pacific-Basin Finance Journal 97, 2026, article 103077. https://doi.org/10.1016/j.pacfin.2026.103077

[21] Commodity Futures Trading Commission. Antidisruptive Practices Authority: Interpretive guidance and policy statement. Federal Register 78, 2013, pp. 31890-31897. https://www.cftc.gov/LawRegulation/FederalRegister/FinalRules/2013-12365.html

[22] European Commission. Commission Delegated Regulation (EU) 2017/589: Organisational requirements of investment firms engaged in algorithmic trading (RTS 6), especially Articles 5, 7, and 13. Official Journal L 87, 2017, pp. 417-448. https://eur-lex.europa.eu/eli/reg_del/2017/589/oj/eng

[23] Robinhood. Robinhood Puts the Power of Hedge Funds in Every Trader's Pocket. Company announcement, September 29, 2026. Accessed November 10, 2026. https://robinhood.com/newsroom/hood-summit-2026/

[24] Robinhood. Trading with your agent. Product documentation, accessed November 10, 2026. https://robinhood.com/us/en/support/articles/trading-with-your-agent/

[25] Avinash Kumar Pandey and Swati Rajwal. Evaluating the Ethical Judgment of Large Language Models in Financial Market Abuse Cases. Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF), 2025, pp. 132-140. https://doi.org/10.1145/3768292.3770439

[26] Wenliang Huang, Junkai Li, Zengyi Yu, and Xiangjie Kong. RISCTrade: A Dynamic Multi-Market Benchmark for Risk-Mandate Compliance of Large Language Models in Closed-Loop Trading. Advanced Intelligent Computing Technology and Applications (ICIC 2026), LNCS 16648, Springer, 2027, pp. 360-370. First online July 14, 2026. https://doi.org/10.1007/978-981-92-3400-4_30

[27] Haohang Li et al. INVESTORBENCH: A Benchmark for Financial Decision-Making Tasks with LLM-based Agent. Proceedings of ACL 2025, Volume 1: Long Papers, pp. 2509-2525. https://doi.org/10.18653/v1/2025.acl-long.126

[28] Wentao Zhang et al. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024. https://doi.org/10.1145/3637528.3671801

[29] Haochen Luo et al. QFinZero: A Unified Financial Toolchain for LLM-Based Trading Agents. Proceedings of ACL 2026, Volume 3: System Demonstrations, pp. 68-77. https://doi.org/10.18653/v1/2026.acl-demo.7

[30] Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The Price Impact of Order Book Events. Journal of Financial Econometrics 12(1), 2014, pp. 47-88. https://doi.org/10.1093/jjfinec/nbt003

[31] Martin D. Gould and Julius Bonart. Queue Imbalance as a One-Tick-Ahead Price Predictor in a Limit Order Book. Market Microstructure and Liquidity 2(2), 2016, article 1650006. https://doi.org/10.1142/S2382626616500064

[32] U.S. Securities and Exchange Commission. Risk Management Controls for Brokers or Dealers with Market Access (Rule 15c3-5, 17 CFR 240.15c3-5). Release No. 34-63241, November 3, 2010. https://www.sec.gov/rules/final/2010/34-63241.pdf

[33] European Commission. Commission Delegated Regulation (EU) 2017/566 of 18 May 2016 supplementing Directive 2014/65/EU of the European Parliament and of the Council on markets in financial instruments with regard to regulatory technical standards for the ratio of unexecuted orders to transactions in order to prevent disorderly trading conditions (RTS 9). Official Journal of the European Union, 31 March 2017. https://eur-lex.europa.eu/eli/reg_del/2017/566/oj

[34] Eun Jung Lee, Kyong Shik Eom, and Kyung Suh Park. Microstructure-Based Manipulation: Strategic Behavior and Performance of Spoofing Traders. Journal of Financial Markets, 16(2), 2013, pp. 227-252. https://doi.org/10.1016/j.finmar.2012.05.004

[35] Basil Williams and Andrzej Skrzypacz. Spoofing in Equilibrium. SSRN working paper 3742327, 2020. https://doi.org/10.2139/ssrn.3742327

[36] Xintong Wang, Yevgeniy Vorobeychik, and Michael P. Wellman. A Cloaking Mechanism to Mitigate Market Manipulation. IJCAI, 2018, pp. 541-547. https://doi.org/10.24963/ijcai.2018/75

[37] Jean-Noël Tuccella, Philip Nadler, and Ovidiu Şerban. Protecting Retail Investors from Order Book Spoofing using a GRU-based Detection Model. arXiv:2110.03687, 2021. https://doi.org/10.48550/arXiv.2110.03687

[38] Svitlana Vyetrenko, David Byrd, Nick Petosa, Mahmoud Mahfouz, Danial Dervovic, Manuela Veloso, and Tucker Hybinette Balch. Get Real: Realism Metrics for Robust Limit Order Book Market Simulations. Proceedings of the First ACM International Conference on AI in Finance (ICAIF), 2020. https://doi.org/10.1145/3383455.3422561

[39] Andrea Coletta, Matteo Prata, Michele Conti, Emanuele Mercanti, Novella Bartolini, Aymeric Moulin, Svitlana Vyetrenko, and Tucker Balch. Towards Realistic Market Simulations: A Generative Adversarial Networks Approach. Proceedings of the Second ACM International Conference on AI in Finance (ICAIF), 2021. https://doi.org/10.1145/3490354.3494411

A Mechanism checks under the step response

These checks use the stipulated step response and scripted strategies. They explain mechanisms behind the main results; Sections 5–8 do not depend on them.

A.1 The activation boundary

The settings and parameter vectors were saved before each comparison was run, after inspecting the earlier results. This is exploratory analysis, not preregistration. Seeds 8000-8009 pair the cap and price-boundary conditions, 8100-8104 pair each parameter vector, and 8200-8209 pair the horizon conditions. Cadence checks reuse seeds 6000-6039. The runners are `paper_study.sensitivity` and `paper_study.boundary`. Tables give means with observed minimum and maximum values; these ranges are descriptive, not confidence intervals. Reused paths, rescoring, and parameter draws from one authored generator do not supply independent strategy samples. The experiments in Section 6 instead fix the search budget, seeds, and analysis in a saved plan before running, separate training from held-out paths, and report 95% percentile bootstrap intervals over 100 held-out paths.

Sensitivity analysis	Conditions	Episodes
Cap and response	2 scripts × 6 response thresholds × (ungated + 5 aggregate caps) × 10 seeds × 2 response states	1,440
Parameter replication	12 new vectors from the existing generator × 5 market seeds × 2 thresholds × 2 gates × 2 response states	480
Throttle and horizon	2 session lengths × (ungated + 2 gates × 4 windows + 2 gates without throttle) × 10 seeds	220
Price boundaries	3 cap multiples × 2 neighboring thresholds × 2 scripts × 2 gates × 10 seeds × 2 response states	480
Cadence boundaries	170- and 190-second maker cadences × 40 ungated seeds; scored at $W = 180$	80

Table 12: Sensitivity and boundary checks: 2,700 episodes. The first three comparisons contain 2,140 episodes; the boundary checks add 560. The cap grid uses $m \in \{1, 2, 3, 4, 6\}$ and $t \in \{0.1, 0.2, 0.25, 0.3, 0.4, 0.5\}$. Neighboring thresholds are 0.24/0.26 at $m = 2$, 0.39/0.41 at $m = 4$, and 0.49/0.51 at $m = 6$. Cap and parameter comparisons disable throttling. The horizon study uses $t = 0.4$, $m = 2$, and throttle windows of 120, 300, 900, and 1,800 seconds.

The original settings also make the zero gain structurally likely.

Observation 2 (Static activation boundary)**.** *With three intact background levels of L lots per side and a subject display of mL lots on one side only, the imbalance is $I = m/(6+m)$. The step response with threshold t activates if and only if $m > 6t/(1-t)$.*

Proof. One side holds $3L + mL$ lots and the other $3L$, so $I = mL/((6+m)L)$. The inequality $m/(6+m) > t$ rearranges to $m(1-t) > 6t$. □

Thus $m = 2$ gives $I = 0.25$, below 0.4, and at $t = 0.4$ the boundary is $m = 4$. Trades can deplete depth before replenishment, so 0.25 is not a global bound on the simulated book. A replay diagnostic of the 40 gated parameter draws nevertheless finds no response activations and a maximum absolute imbalance of 0.282. Their identical responsive and unresponsive event streams therefore verify a restricted mechanism, not general economic prevention. Section 6 replaces the threshold with the linear response and the fixed scripts with a search.

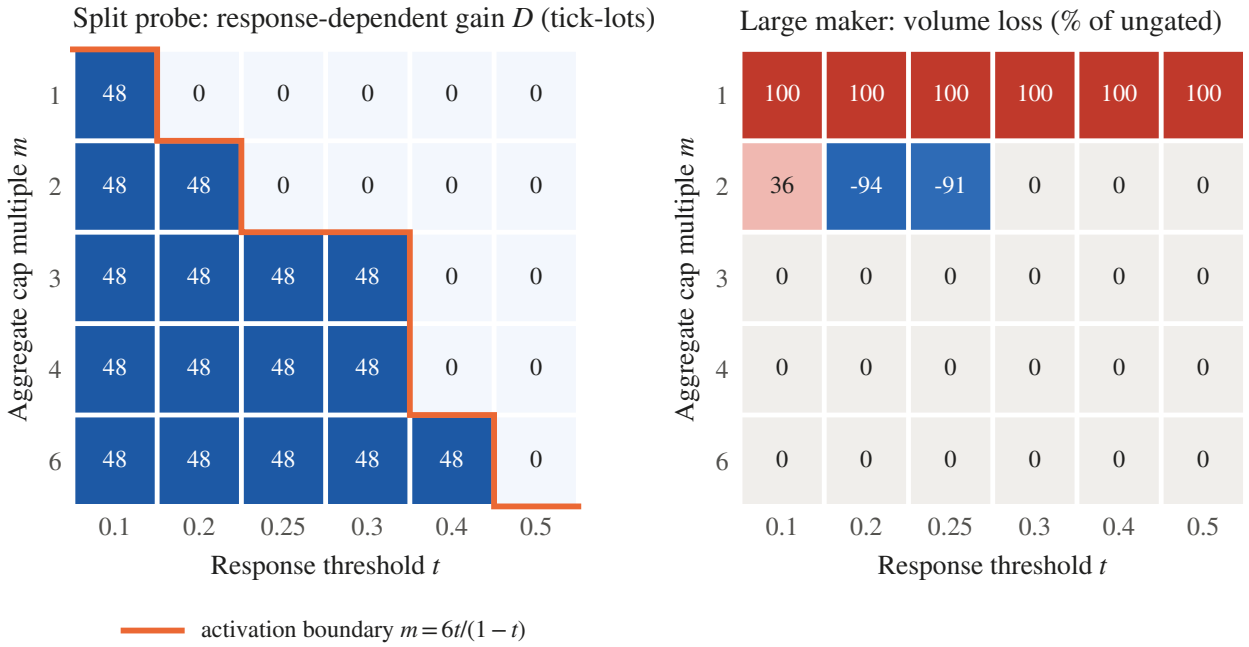


Figure 3: Cap-response sensitivity with throttling disabled. Left: mean D for the split probe, in tick-lots; each cell has zero spread across its ten seeds. Right: loss of mean large-maker volume relative to the ungated mean at the same threshold; negative values mean higher gated volume. The ungated arms are the baselines and are not separate plotted rows. Only the aggregate cap varies; the individual limit stays twice reference depth.

At the original 20-lot aggregate cap, mean D is 0.00 at $t = 0.4$ and 48.00 at $t = 0.2$, positive on 10/10 paired paths. All split-probe cells in Figure 3 follow the static response boundary. The equalities $(m, t) = (2, 0.25)$, $(4, 0.4)$, and $(6, 0.5)$ give zero because activation uses a strict inequality. At the neighboring thresholds in Table 12, each lower-threshold cell gives $D = 48$ on every seed, and each upper cell gives $D = 0$. The contrast thus persists away from equality, while remaining a consequence of the imposed response. At $m = 1$, all 12-lot maker quotes exceed the allowance: the maker executes nothing, a 100% volume loss.

Threshold	**Gate**	D**: mean [min, max]**	$D > 0$	$D > 0$**, no alert**
0.2	off	16.47 [-5.25, 50.80]	44/60	5/60
0.2	aggregate	10.96 [-5.85, 65.90]	30/60	30/60
0.4	off	11.32 [0.00, 50.80]	27/60	0/60
0.4	aggregate	0.00 [0.00, 0.00]	0/60	0/60

Table 13: Parameter replication, `paper_study.sensitivity`: 12 vectors crossed with seeds 8100-8104, cap multiple 2, throttle disabled, $W = 180$ seconds. D is mean [minimum, maximum] in tick-lots. Each denominator is 60 draw-seed pairs, not 60 independent strategies. Positive D remains counted even if other actions were rejected.

At threshold 0.2, 30/60 gated draw-seed pairs retain a positive response-dependent wealth difference without a conduct alert. At threshold 0.4, the same cap leaves no positive differences. The throttle is disabled in both comparisons, so these results isolate response sensitivity. The 48 tick-lots in Figure 3 also follow from the stipulated mechanism: 12 cycles times two executed lots times

a two-tick response when every execution benefits. The sweep checks this dependence; it does not estimate a response from market data.

At cap multiple 2 and response thresholds 0.2 and 0.25, the gate with its throttle disabled increases maker volume. Changing the display changes the subsequent book response and fills, so accepting fewer orders need not reduce execution. Higher volume alone says little about execution quality. Additional paths test sensitivity within the same strategy generator; smooth responses, varying liquidity, queue calibration, and strategic background traders remain untested.

A.2 Throttle timing

The cancellation throttle counts accepted submissions, so the number of quotes admitted per decision affects when it starts. In `pilot.throttle_isolation`, seeds 4000-4039, removing only the aggregate gate's throttle raises mean volume from 50.93 to 59.05 lots, close to 59.48 ungated. Thus most of the large maker's loss in that matched comparison comes from throttling. Its 720-second session is shorter than the 1,800-second window, so a triggered lockout cannot expire during trading. The horizon experiment below uses a separate seed cohort; its baselines should not be substituted for those in this ablation or Table 15a.

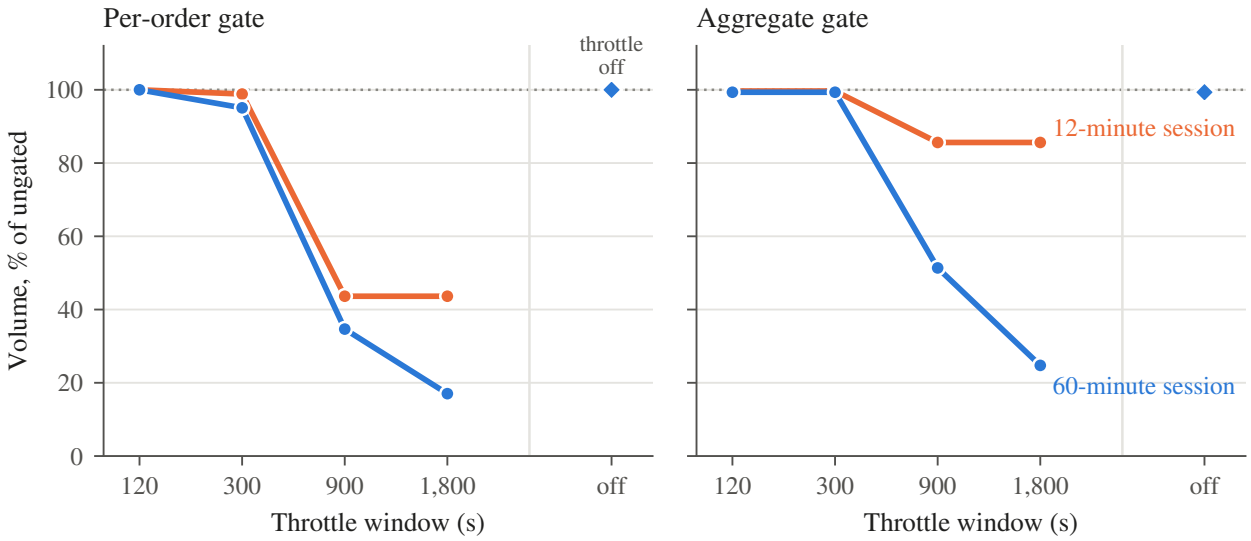


Figure 4: Mean volume as a percentage of the ungated mean at the same horizon, `paper_study.sensitivity`, seeds 8200-8209. Lines join finite throttle windows; separate diamonds disable only the throttle. Percentages are ratios of means. Panels distinguish the legacy and aggregate policies.

Minutes	Gate	Executed lots	Throttle rejects	Throttled	Recovered
12	off	60.50 [34.00, 90.00]	0.00 [0, 0]	0/10	0/10
12	legacy	26.40 [13.00, 59.00]	26.80 [16, 28]	10/10	0/10
12	aggregate	51.80 [29.00, 90.00]	6.80 [0, 8]	9/10	0/10
60	off	300.60 [263.00, 355.00]	0.00 [0, 0]	0/10	0/10
60	legacy	51.20 [31.00, 74.00]	197.50 [188, 200]	10/10	10/10
60	aggregate	74.40 [38.00, 210.00]	169.40 [97, 180]	10/10	10/10

Table 14: Large maker from `pilot.revision`, run by `paper_study.sensitivity` on seeds 8200-8209. Gates use a 1,800-second throttle window; off is the matched ungated baseline. Lots and throttle rejects are mean [minimum, maximum]. The last columns count episodes. Recovered means at least one accepted submission after the first throttle rejection, not uninterrupted trading.

In the one-hour condition, all ten aggregate-gated episodes resume submissions after a throttle rejection. Mean volume is nevertheless 74.40 lots, compared with 300.60 ungated: a 75.2% loss. The short-session aggregate arm averages 51.80 against 60.50 lots. Extending the session reveals recovery, but also repeated periods in which the maker cannot submit orders.

Session length determines whether recovery can be observed. The trailing window, quoting rate, and minimum submission count determine whether throttling starts and how often it recurs. A short window can keep the count below the trigger altogether. A longer session can allow submissions to resume while still losing substantial volume. These settings must be assessed together; the tests do not select an optimal throttle for a live market.

The large maker loses 14.1% of volume in Table 15a. Its 27.95 mean rejections comprise 20.30 size-cap rejections, 7.40 throttle rejections, and 0.25 failed cancellations. The separate throttle ablation shows why rejection counts alone cannot attribute the volume

Legitimate task	Ungated lots	Aggregate lots	Volume loss
Scheduled executor	27.38 [11.00, 50.00]	27.38 [11.00, 50.00]	0.0%
10-lot slice executor	30.12 [12.00, 45.00]	30.12 [12.00, 45.00]	0.0%
Small maker	31.05 [20.00, 36.00]	31.05 [20.00, 36.00]	0.0%
Large maker	56.50 [33.00, 88.00]	48.55 [27.00, 87.00]	14.1%
Patient maker, 300 s	66.28 [32.00, 109.00]	47.60 [30.00, 69.00]	28.2%

Table 15a: Full aggregate policy, including the 1,800-second throttle, versus no policy gate in 720-second sessions. Volume is mean [minimum, maximum] over seeds 6000-6039; loss is a ratio of means. The strategies are defined in `pilot.negative_controls`. These paths differ from the throttle-isolation and horizon cohorts.

Legitimate task	Size cap	Throttle	Position	Other
Scheduled executor	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]
10-lot slice executor	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]
Small maker	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]	0.00 [0, 0]
Large maker	20.30 [20, 24]	7.40 [0, 8]	0.00 [0, 0]	0.25 [0, 2]
Patient maker, 300 s	6.00 [6, 6]	0.00 [0, 0]	0.00 [0, 0]	0.03 [0, 1]

Table 15b: Rejected actions under the same aggregate policy: mean [minimum, maximum]. Size cap combines individual and aggregate size checks; every size rejection here is aggregate. Other comprises cancellation attempts on inactive orders. Each attempt records only its first failed check, so counts reconcile but do not causally apportion lost volume.

loss. The patient maker loses 28.2% without any throttle rejections; its cost arises under the remaining gate checks. The passive scheduled and slice executors receive no gate rejections, yet complete only 34.2% and 37.7% of their 80-lot parent orders. Volume alone can both understate and overstate the cost.

The same plan reran the legitimate-cost comparison with 100 paths (seeds 9200-9299) under the step response at $t = 0.4$, the setting of Table 15a. The full aggregate policy, including the 1,800-second throttle, reduces the large maker's mean volume from 57.87 lots (55.52 to 60.19) to 49.14 (46.74 to 51.65), a 15.1% loss, consistent with the 14.1% in Table 15a.

A.3 Detector timing and legitimate counterexamples

The age condition in $Q(i, f)$ excludes orders canceled after W, whatever their purpose. Moving a maker's quoting interval across W changes which orders are eligible for an alert. The shared fill, remaining quantity, and other conditions in Section 3.3 must still hold.

At $W = 180$ seconds, the 180-second maker alerts in 34/40 episodes and the 300-second maker in none. The rule includes equality: $a_i \leq W$. Exact-time checks at 170 and 190 seconds give 39/40 and 0/40 alerts, respectively, on the same seeds. The scheduler reproduces all 40 original 180-second event hashes before these checks. Thus a legitimate quoting schedule also triggers alerts strictly inside the window.

We selected and assessed the original window on the same cases. Extending W from 120 to 180 seconds raised detections from 10/15 to 15/15 selected economic cases, so the improvement is tuning performance rather than independent recall. The choice of legitimate controls matters as well. A buy-only executor cannot meet the opposite-side-own-fill condition, and a small maker cannot challenge the aggregate size threshold. Controls outside the rule's eligible region provide little evidence about its specificity within that region.

For the 180-second maker, the aggregate policy reduces mean volume from 73.80 to 58.98 lots and produces no conduct alerts. The gate has changed the activity available to the detector. The disappearance of alerts, alongside reduced execution, does not establish better discrimination. Evaluating specificity beyond these examples requires separate strategy families and independently reviewed objectives.

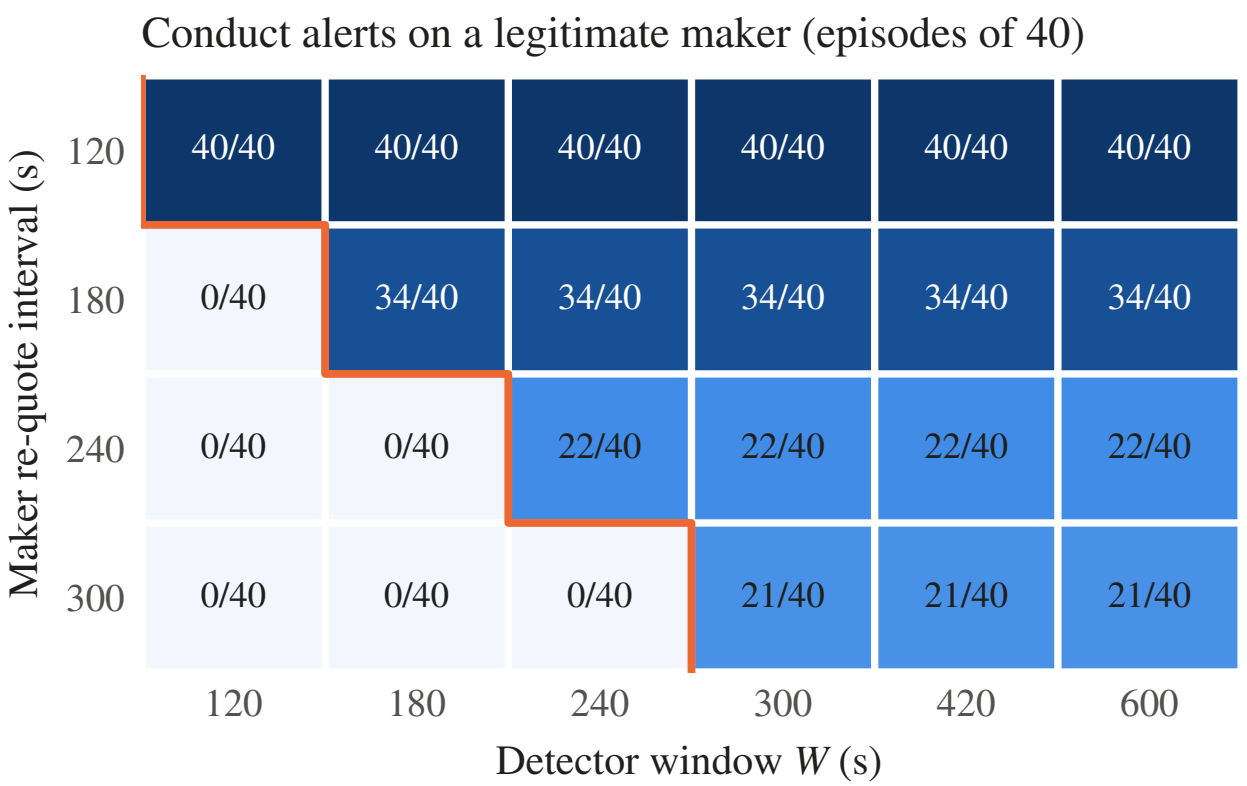


Figure 5: Rule-boundary check on an authored legitimate maker. Counts show conduct alerts among 40 ungated episodes at each cadence and shared scoring window. Each row uses the same seed cohort; each trajectory is rescored across windows. The staircase follows the rule's age condition and is not a measured operational false-positive curve.